

MINOR-2 PROJECT

Software Requirements Specification

For

Damage Object Detection on a Conveyor belt(Box)

6-03-2025

Submitted By

Specialization	SAP ID	Name
BTECH-CSE(AIML-NH)	500109512	Tushar Kukreja
BTECH-CSE(AIML-NH)	500106999	Manan Kumar
BTECH-CSE(AIML-NH)	500109501	Shivanshu
BTECH-CSE(AIML-NH)	500107998	Suryansh Chauhan



UNIVERSITY WITH A PURPOSE

Department of Informatics

School Of Computer Science

UNIVERSITY OF PETROLEUM & ENERGY STUDIES,
DEHRADUN- 248007. Uttarakhand

Project Guide
Dr. Shubranil Das

Table of Contents

Topic		Page No
Table of Content		1
1	Introduction	2
	1.1 Purpose of the Project	2
	1.2 Target Beneficiary	2
	1.3 Project Scope	3
	1.4 References	
2	Project Description	3-6
	2.1 Reference Algorithm	3
	2.2 Data/ Data structure	3
	2.3 SWOT Analysis	4
	2.4 Project Features	4
	2.5 User Classes and Characteristics	5
	2.6 Design and Implementation Constraints	5
	2.7 Design diagrams	6
	2.8 Assumption and Dependencies	6
3	System Requirements	6-7
	3.1 User Interface	6
	3.2 Software Interface	7
	Non-functional Requirements	7
	4.1 Performance requirements	7
4	4.2 Security requirements	8
	4.3 Software Quality Attributes	8
	Other Requirements	8

1. Introduction

With the rise of automation in industries, the need for efficient and accurate quality control mechanisms has become crucial. Detecting damages in products such as boxes moving on a conveyor belt manually is both labor-intensive and prone to errors. This project aims to automate the damage detection process using **YOLO (You Only Look Once)**, a real-time object detection algorithm, to identify damaged boxes with high precision. The project will also employ containerization using **Docker** for easy deployment and scaling across different systems.

1.1.Purpose of the Project

The purpose of this project is to develop an automated system that detects damaged boxes on a conveyor belt using computer vision. By leveraging the **YOLO model**, the system will identify and flag damaged boxes in real-time, improving quality control and reducing labor costs. A **Docker file** will be created to ensure the system is portable and scalable across different environments.

1.2.Target Beneficiary

The primary beneficiaries of this project are industries and manufacturing units that rely on conveyor belts for product transportation and packaging. This includes:

- **Manufacturing companies:** to ensure that only undamaged products reach the end consumer.
- **Logistics and supply chain managers:** for improving product quality checks in warehouses.
- **Quality control teams:** who can automate the inspection process, reducing manual effort and errors.

1.3.Project Scope

The scope of this project includes:

- Implementing a **YOLO-based object detection model** to identify damaged boxes in real-time.
- Developing a **Docker container** to package the model and necessary software for easy deployment.
- Ensuring compatibility with various conveyor belt speeds and lighting conditions.
- Providing an intuitive user interface for monitoring and reviewing detected damages.
- Integrating the system with existing production pipelines to automate the quality control process.

2. Project Description

The **Box Damage Detection on a Moving Conveyor Belt** project uses computer vision techniques to identify damaged boxes in real-time. The system is powered by the **YOLO (You Only Look Once)** model, known for its efficiency in detecting objects in real-time. The project also employs containerization using **Docker** to ensure ease of deployment across different systems.

2.1.Reference Algorithm

The reference algorithm for this project is the **YOLO (You Only Look Once)** model. YOLO is a real-time object detection system that processes the entire image in one go, making it fast and efficient. The algorithm breaks down the image into a grid, where each grid cell predicts bounding boxes and probabilities for potential damage. The high-speed processing of YOLO allows for real-time detection on a moving conveyor belt.

2.2.Data/ Data Structure

- **Image Data:** The primary data consists of images or video frames captured from the conveyor belt. Each image contains multiple boxes that need to be inspected for damage.
- **Labels:** The images are labeled with bounding boxes around areas of detected damage (or no damage).
- **Data Preprocessing:** The data undergoes resizing, normalization, and augmentation to ensure the model is trained for different conditions like lighting or angles.
- **Data Storage:** The image data and model will be stored in a cloud-based or local storage system, with structured labels for training and evaluation.

2.3.SWOT Analysis

- **Strengths:**
 - Real-time detection using a highly efficient YOLO algorithm.
 - Automated process reduces human error and manual effort.
 - Scalable through Docker containerization.
- **Weaknesses:**
 - Performance may vary under extreme lighting conditions.
 - Initial setup requires high computational power for model training.
- **Opportunities:**
 - Can be extended to detect other forms of product damage or defects.
 - Potential integration with other industrial IoT systems for full automation.
- **Threats:**
 - High competition from other industrial automation solutions.

- Dependence on hardware quality for capturing clear images.

2.4.Project Features

- **Real-time Damage Detection:** The system uses the YOLO model to detect damage on boxes in real-time as they move on the conveyor belt.
- **Docker Container:** The entire system is containerized using Docker, allowing for easy deployment and scaling in different environments.
- **Logging and Reporting:** Keeps a record of all detections with time stamps and images for further review and quality assurance.

2.5.User Classes and Characteristics

- **Factory Operators:** Responsible for monitoring the system and acting upon detected damage.
- **Quality Control Engineers:** Use the system to automate the detection of damaged boxes and review logs for further analysis.
- **System Administrators:** Manage system deployment, updates, and Docker containers to ensure smooth operations.
- **Developers:** Responsible for maintaining the codebase and ensuring model accuracy over time.

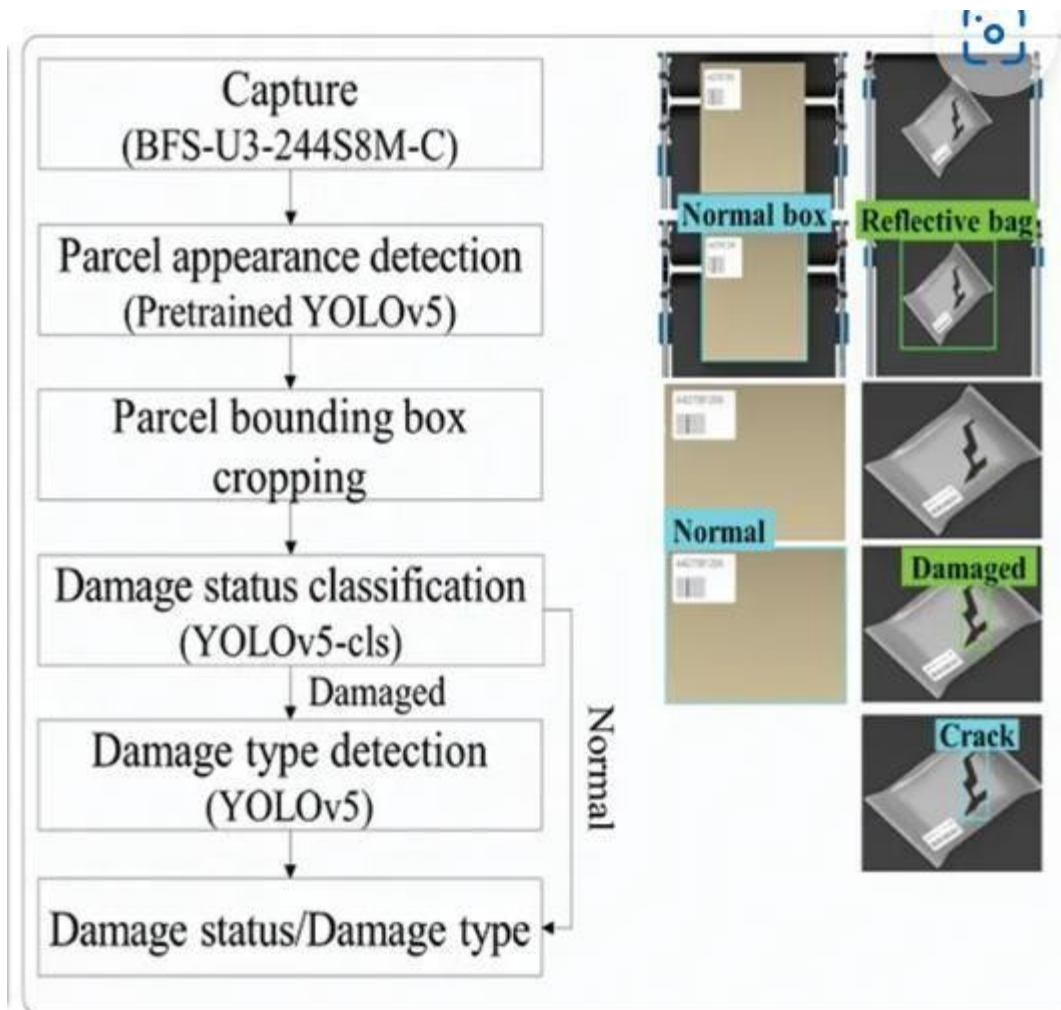
2.6.Design and Implementation Constraints

- **Hardware Constraints:** Requires cameras with sufficient resolution to capture moving boxes on the conveyor belt and a machine with a capable GPU for running the YOLO model in real-time.
- **Timing Requirements:** The system should be able to process each frame within a second to detect damages on fast-moving conveyor belts.
- **Memory Constraints:** The system needs to efficiently handle high volumes of image data without exceeding memory limits.
- **Software Constraints:** The model will run in a **Docker** container to ensure cross-platform compatibility.

2.7.Design Diagrams

The design includes:

- **System Architecture Diagram:** Showing the flow from image capture on the conveyor belt, passing through the YOLO model for damage detection, and finally logging results and issuing alerts.
- **Data Flow Diagram:** Illustrates the flow of image data through preprocessing, detection, and post-processing stages.



(You can provide visual diagrams like System Architecture and Data Flow Diagram here.)

2.8. Assumptions and Dependencies

- **Assumptions:**
 - The conveyor belt operates at a consistent speed for proper image capturing.
 - Cameras used for image capture have sufficient resolution and frame rate.
 - Docker will be installed on all target machines for deployment.
- **Dependencies:**
 - The system relies on high-quality camera equipment for clear image capture.
 - The model depends on sufficient and labeled training data to maintain detection accuracy.
 - Stable internet connection may be required if the system is integrated with cloud storage or reporting systems.

3. System Requirements

The system for **Box Damage Detection on a Moving Conveyor Belt** must meet certain user interface and software interface requirements to function efficiently in an industrial environment.

3.1. User Interface

- **Real-time Detection Display:**
A live video feed from the conveyor belt will be displayed, with bounding boxes around detected damages, along with an overlay indicating whether each box is damaged or not.
- **Alerts Section:**
When a damaged box is detected, the system will trigger a visual alert on the dashboard (e.g., red highlight) along with an option for sound alerts. The alert system can be customized by the user.
- **Reports and Logs:**
A section for reviewing previous damage detection logs, with the ability to view detailed information such as the image captured, timestamp, and the type of damage identified.
- **Settings Page:**
Users will be able to configure the sensitivity of the YOLO model for damage detection, manage Docker settings, and adjust the parameters for the conveyor belt's speed and camera resolution.

3.2. Software Interface

- **YOLO Model Integration:**
The system will integrate the **YOLO object detection model** through a trained neural network. The software interface will link the incoming video feed to the YOLO detection system for real-time processing.
- **Docker Integration:**
The application will be containerized using **Docker** to ensure platform independence and ease of deployment. The Docker container will include the YOLO model, necessary dependencies, and any additional tools required for image processing and detection.
- **Camera Input:**
The system will interface with cameras placed above the conveyor belt. The software will support multiple camera inputs and handle video streams efficiently.
- **Alert System Integration:**
The software will connect with external alert systems (e.g., sound alarms, light indicators) to notify operators in case of damage detection.

- **Building and Pushing Images:** Once the image is built locally, it can be pushed to AWS Elastic Container Registry (ECR), making it accessible for deployment.
- **AWS ECS/EKS for Deployment:**
- **ECS (Elastic Container Service):** Orchestrates Docker containers, scaling based on load.
- **EKS (Elastic Kubernetes Service):** Offers container management through Kubernetes, handling complex scaling and orchestration.

4. Non-Functional Requirements

4.1. Performance Requirements

- **Response Time:** The system should detect damages and provide feedback within **2-5 seconds** of image capture to ensure real-time monitoring.
- **System Uptime:** The system should maintain an uptime of **99%** to ensure continuous operation in a production environment.

4.2. Security Requirements

- **User Authentication:** Implement robust user authentication mechanisms, including **username/password** and **two-factor authentication** to prevent unauthorized access.
- **Data Encryption:** All sensitive data, including user credentials and detection logs, must be encrypted in transit and at rest to safeguard against data breaches.
- **Access Control:** Role-based access control should be employed to restrict user access to sensitive features and data based on their roles.

4.3. Software Quality Attributes

- **Reliability:** The system should consistently perform detection tasks accurately under various operational conditions.
- **Scalability:** The architecture should allow for easy scaling to accommodate additional cameras or conveyor belts without significant reconfiguration.
- **Maintainability:** The system's codebase should adhere to coding standards and best practices, making it easy to update and maintain over time.
- **Usability:** The user interface must be intuitive, allowing users to navigate the system efficiently without extensive training.

5. Other Requirements

5.1.Documentation

- User manuals and technical documentation for system setup and operation.

5.2.Training and Support

- Training materials for operators and technical support for maintenance.